Conversion

Question: Can we Convert from one Data type to another ?.

char
↓
byte → short → int → long → double

Conversion without loss

only in one direction

int → float
long → double
} loss of information

Operators

operands

one of them double → double ✓
float → float ✓
long → long ✓
otherwise int

} orders of conversion when doing operations

cast :-

conversion in which loss of information is allowed

int → long (no loss of info)

long → int ✗ loss of info

```
long x = 32
int y = (int) x;
```

(cast)

double x = 9.97

int y = (int) x → In most cases rounds

int y = (int) Math.round(x)

If you try to cast a number to a range out of target type → diff value

```
int x = 128
byte y = (byte) x
```

-128 to 127

y → -128

(*) In conversion you cannot convert boolean with int like C/C++

Assignment operator

Implicit casting	Explicit casting
No loss of value	Loss of value
	Explicit

byte
↓
short → int → long
↑
char

float → double

But compound Assignment operator actually bypasses Explicit Typecasting

```
byte b = 5;
b = b + 5;   X   error
b += 5;      ✓   work
b = (byte)(b+5);
```

Increment / Decrement

++a a++

--a a--

For eg:-

a = 10

s.o.p(++a) s.o.p(a++)

↓ ↓

a incremented to 11 printed printed 10 ✓

printed 11 a is then 11

Relational

↓

AND && ! precedes

OR || &&

NOT ! ||

Conditional

operator selects value based on Boolean Expression

condition ? Expression1 : Expression 2

age >= 18 ? "Adult" : "young"

SWITCH EXPRESSIONS [JAVA 14]

```
String seasonName = switch (seasonCode) {
    case 0 -> "spring";
    case 1 -> "summer";
    case 2 -> "winter";
    default -> "???";
};
```

case labels -> Strings or constants of enumeration types or integers

- You can use multiple labels

```
case 0, 1, 2 -> "this"
```

Enumeration

```
Size = enum { "SMALL", "MEDIUM", "LARGE" }
```

```
String label = switch (itemSize) {
    case "SMALL"
    case "MEDIUM"
    case "LARGE"
};
```

Size =

↓

No default -> defined set of choices =

Switch expression

Integer String

Default values

Bitwise operators

Operates on bits level

12

↓

you want to know bits

2 | 12 0
2 | 6 0
2 | 3 1
1

12 -> 1100

Bitwise -> bits

8

12 & 13

✓

1 1 0 0
8 4 2 1

(Bitwise)

& | ^ ~

And or xor not

12 -> 1100
13 -> 1101
12 1100

✓

bit from ~~left~~ right

= (ns 0b100)/0b000

When applied to boolean values & and | operators → boolean value

Short circuit fashion

1) (a == b && b == c)
↓
False Then b == c will not evaluated

2) (a == b || b == c)
↓
True Then b == c is not evaluated

But Bitwise operators
Evaluate both before result is completed

Shift operators

>> → right shift

<< → left shift

n >> 3

12 → 1100

0000 1100

>>2 Right side shift

0000 0011

3

<< 2 ✓

0000 1100

0 0110000

32 16 8 4 2 1

= 48

✓

a >>>

↓

fills top bits with zero

(*) There is no <<< operator

32
16
8
4
8

Right hand argument is reduced to modulo 32 unless left hand argument is long

long is right hand

↓

modulo 64

Operator precedence

Left → right

[] . ()

. Dot operator
() Method call

! ++ -- (+) (-) () (cast) new Right → left
unary unary

* % / left → right

+ - left → right

<< >> >>> left → right

< <= > >=
instanceof left → right

== != left → right

&
^
|] left → right

&&
||] Left → right

?:] Right → left

= += Assign